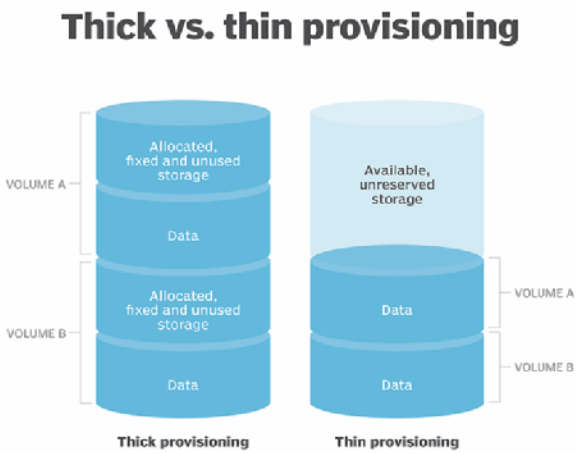
BLOQUE 4 – TEMA 3 – LABORATORIO VIRTUALIZACION



Comandos de Docker

* #### docker info (informacion sobre el uso de docker en el equipo)

  Containers: 7
  Running: 0
  Paused: 0
  Stopped: 7
  Images: 5
  ....
  Registry: https://index.docker.io/v1/ → registry por defecto.
  Docker Root Dir: /var/lib/docker

* #### docker pull imagen (descarga una imagen del hub) → DockerHub

  docker pull jenkins

* #### docker version (version del runtime docker)
* #### docker ps (muestra un listado de los contenedores activos del sistema)

  docker ps -a (muestra todos los contenedores, activos e inactivos)

* #### docker run imagen (arrancar un nuevo contenedor en base a una imagen y devuelve su identificador ≈ hash)

  docker run -p 8080:8080 -p 50000:50000 jenkins (arranca el contenedor con esos mapeos de puertos HOST:CONTAINER)
  el primer puerto es el real y el siguiente es el de Docker. PAT: Port Adress Translation.

  docker run centos -it /bin/bash (levanta un contenedor con CentOS en modo interactivo y al finalizar nos saldrá el shell)

  otras opciones interesantes:
  -v /usr/local/bin:/target (asocia un volumen con un almacenamiento externo. Formato PATH_HOST:PATH_CONTAINER)
  -d (arranque en segundo plano)
  -i (interactivo)
  -t tty

* #### docker images → muestra todas las imágenes que tengo y algo de su información.
* #### docker image ls ≈ docker images
* #### docker create NombreImagen (crea un contenedor pero no lo arranca)
* #### docker start/stop id_del_contenedor (arrancar y parar un contenedor)
* #### docker build -t NombreImagen:TagImagen (crea una imagen a partir del fichero Dockerfile situado en el directorio actual)
* #### docker attach → enlaza la salida, la entrada y el error al contenedor. Permite conectarnos al contenedor.
* #### docker kill id_contenedor → hace el stop de forma inmediata.
* #### docker export → guarda un contenedor en un tar, hace un backup.
* #### docker commit → crea una imagen a partir de un contenedor ya en ejecución.

example Dockerfile for https://docs.docker.com/engine/examples/postgresql_service/

```
FROM ubuntu:16.04
# Add the PostgreSQL PGP key to verify their Debian packages. It should be the same key as https://www.postgresql.org/media/keys/ACCC4CF8.asc
RUN apt-key adv --keyserver hkp://p80.pool.sks-keyservers.net:80 --recv-keys B97B0AFCAA1A47F044F244A07FCC7D46ACCC4CF8
# Add PostgreSQL's repository. It contains the most recent stable release of PostgreSQL, ``9.3``.
RUN echo "deb http://apt.postgresql.org/pub/repos/apt/ precise-pgdg main" > /etc/apt/sources.list.d/pgdg.list
# Install ``python-software-properties``, ``software-properties-common`` and PostgreSQL 9.3. There are some warnings (in red) that show up
# during the build. You can hide them by prefixing each apt-get statement with DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y python-software-properties software-properties-common postgresql-9.3 postgresql-client-9.3 postgresql-
contrib-9.3
# Note: The official Debian and Ubuntu images automatically ``apt-get clean``
# after each ``apt-get``
# Run the rest of the commands as the ``postgres`` user created by the ``postgres-9.3`` package when it was ``apt-get installed``
USER postgres
# Create a PostgreSQL role named ``docker`` with ``docker`` as the password and then create a database `docker` owned by the ``docker`` role.
# Note: here we use ``&&\`` to run commands one after the other - the ``\`` allows the RUN command to span multiple lines.
RUN    /etc/init.d/postgresql start &&\
   psql --command "CREATE USER docker WITH SUPERUSER PASSWORD 'docker';" &&\
   createdb -O docker docker
# Adjust PostgreSQL configuration so that remote connections to the database are possible.
RUN echo "host all  all    0.0.0.0/0  md5" >> /etc/postgresql/9.3/main/pg_hba.conf
# And add ``listen_addresses`` to ``/etc/postgresql/9.3/main/postgresql.conf``
RUN echo "listen_addresses='*'" >> /etc/postgresql/9.3/main/postgresql.conf
# Expose the PostgreSQL port
EXPOSE 5432/tcp
# Add VOLUMEs to allow backup of config, logs and databases
VOLUME  ["/etc/postgresql", "/var/log/postgresql", "/var/lib/postgresql"]
# Set the default command to run when starting the container
CMD ["/usr/lib/postgresql/9.3/bin/postgres", "-D", "/var/lib/postgresql/9.3/main", "-c", "config_file=/etc/postgresql/9.3/main/postgresql.conf"]
```

Arrancas el servicio de postgres 1º param 2º param 3º param 4º param

---

FROM: imagen:tag base (ubuntu,alpine,... a partir de la cual crear la nuestra)
ENV: definir variables de entorno. Existe durante la construcción de la imagen y en el contenedor durante su ejecución.
ARG: crea variables que solo se usan durante la creación, no durante la ejecución.
COPY: copiar desde el HOST/url hacia dentro de la imagen. Copias en la imagen cuando se crea, no cuando se ejecuta.
RUN: ejecución de comandos: apt, mkdir, etc.
EXPOSE: exponer puertos en el CONTAINER. Solo con esto no podremos acceder desde el HOST.
VOLUME: nos permite definir un directorio cuya información será persistente.
WORKDIR: establece el directorio de trabajo.
USER: usuario con el que se ejecutarán las acciones, por defecto será root.
CMD: comando+argumentos por defecto que se ejecutará cuando se inicie un CONTAINER con esta imagen. Solo puede haber un CMD en el Dockerfile. Aquí colocamos el típico comando que queremos que se ejecute cuando se levante el contenedor. Haciendo un docker run podríamos sobrescribir lo que venga en CMD:

…
CMD [“echo”, ”Hola Mundo”]
…

docker run prueba → mostraría “Hola Mundo”. docker run prueba df → no mostraría “Hola Mundo”, sino que haría un df.

ENTRYPOINT: no se puede sobrescribir, pero CMD sí. El resto es igual que CMD.

---

* #### docker images (mostrar las imágenes que tenemos disponibles)

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
centos	latest	589dc4d40385	41 hours ago	237MB
postgres	latest	ee227fd504c8	3 months ago	348MB

* #### docker exec (ejecutar un comando contra un contenedor en ejecucion)
  docker exec -it centos bash (entramos en el bash del contenedor)
* #### docker attach id_del_contenedor (uniser a un contenedor en ejecucion)
  docker attach 07b0b6f434fe
* #### docker rm id_del_contenedor (borrar contenedor)
  docker rm 9f215ed0b0d3
* #### docker rmi id_de_la_imagen (borrar imagenes)
  docker rmi 7a86f8ffcb25
* #### kubectl (manejar cluster de contenedores sobre una infraestructra de nodos distribuida)

definicion_pods.yaml

```
# Número de versión del api que se quiere utilizar
apiVersion: v1
# Tipo de fichero que se va a crear.
kind: Pod
# Aquí van los datos propios del pod como el nombre y los labels que tiene asociados para seleccionarlo
metadata:
  name: my-nginx
  # Especificamos que el pod tenga un label con clave "app" y valor "nginx"
  labels:
    app: nginx
# Contiene la especificación del pod
spec:
# Aquí se nombran los contenedores que forman parte de este pod. Todos estos contenedores serían visibles por localhost
  containers:
    - name: nginx
      image: nginx
      ports:
        - containerPort: 80
# Aquí se define la política de restauració en caso de que el pod se detenga o deje de ejecutarse debido a un fallo interno.
  restartPolicy: Always
```

kubectl **apply** -f definicion_pods.yaml (modo **declarativo**)
kubectl **create** -f definicion_pods.yaml (modo **imperativo**)
kubectl run my-nginx --image=nginx --port=80 (creacion de un pod sin yaml)
kubectl get pods (pods que están ejecutandose)
kubectl describe pod nombreDelPod (info mas detallada del pod)
kubectl get nodes (nodos en el cluster)
kubectl logs my-nginx

* Cluster: Conjunto de máquinas físicas o virtuales y otros recursos utilizados por kubernetes.
* Nodo: Una máquina física o virtual ejecutándose en kubernetes donde pods pueden ser programados.
* Pod: Son la unidad más pequeña desplegable que puede ser creada, programada y manejada por kubernetes. Este Pod es un grupo de uno o más contenedores, con almacenamiento compartido entre ellos.
* Replication Controller: Se asegura de que el número específicado de réplicas del pod estén ejecutándose. Permite escalar de forma facil los sistemas y maneja la re-creación de un pod cuando ocurre un fallo.
* Service: Es una abstracción que define un conjunto de pods y la lógica para acceder a los mismos.